SteamOS LLM Terminal Interface

Full Product Requirements Document (PRD) + System Design Specification (SDS)

Version: 1.0

Target Platform: Steam Deck / SteamOS Game Mode

Primary Stack: Golang + Bubble Tea + Lipgloss + Alacritty + Ollama

Executive Summary

This project aims to build a native-feeling AI terminal interface for SteamOS Game Mode that allows a Steam Deck to operate a fully featured LLM-powered terminal UI entirely through Steam Input mappings.

The application combines:

- Bubble Tea state management

- Lipgloss ANSI styling

- GPU accelerated terminal rendering via Alacritty

- Local or remote LLM orchestration

- Steam Input controller abstraction

- SteamOS Game Mode compatibility

The result should feel like a first-class cyberpunk terminal operating environment rather than a standard chatbot.

Product Vision

Deliver a fullscreen immersive AI terminal experience optimized for:

- handheld usage

- low-latency interaction

- controller-driven navigation

- retro terminal aesthetics

- developer productivity

- offline-capable AI workflows

The system should support:

- conversations

- code generation

- scripting

- terminal macros

- exportable sessions

- persona switching
- streaming token rendering

Product Goals

Primary Goals

1. Native Steam Deck UX
2. Full controller support
3. Fast startup time
4. Stable streaming UI
5. Offline-first capability
6. Minimal RAM overhead
7. GPU accelerated rendering
8. Fully keyboard fallback compatible

Non-Goals

- Browser-based frontend
- Electron dependency
- Heavy desktop UI frameworks
- Mouse-first interaction design

Core Features

Chat System

- streaming responses
- markdown rendering
- ANSI syntax highlighting
- code block formatting
- persistent history
- token streaming

Persona Engine

- switchable system prompts
- saved personalities
- contextual presets
- workflow modes

Session Management

- save conversations
- export markdown
- import context
- reset thread
- regenerate responses

Steam Deck Integrations

- Steam Input mappings
- radial menu support
- SteamOS keyboard compatibility
- gamescope fullscreen launching

User Personas

Solo Developer

Uses the Deck as a portable coding assistant.

Power User

Uses local LLMs offline while traveling.

Cyberpunk TUI Enthusiast

Values immersive retro terminal aesthetics.

Accessibility User

Requires full controller-only operation.

Technical Architecture

High-Level Stack

SteamOS Game Mode

↓

Gamescope

↓

Launch Script

↓

Alacritty

↓

Bubble Tea Application

↓

LLM Service Layer

↓

Ollama / Remote APIs

Component Breakdown

Renderer Layer

Responsible for:

- layout
- ANSI rendering
- viewport composition
- animation frames
- chroma effects

Input Layer

Responsible for:

- keyboard events
- controller mappings
- Steam Input macros
- clipboard integration

State Layer

Responsible for:

- message history
- application state
- async requests
- loading indicators

Networking Layer

Responsible for:

- streaming inference
- retry logic
- cancellation
- provider abstraction

Persistence Layer

Responsible for:

- local configs
- session exports
- history cache
- theme files

Steam Input Mapping Spec

Steam Deck Control	Keyboard Mapping	Action
A	Enter	Submit
B	Esc	Cancel
X	Backspace	Delete
Y	Space	Insert space
L1	Tab	Focus switch
R1	Shift+Tab	Reverse focus
L2	Ctrl+U	Clear line
R2	Ctrl+V	Paste
L4	Ctrl+P	Cycle persona
R4	Ctrl+R	Regenerate
L5	Ctrl+S	Save session
R5	Ctrl+N	New thread

Bubble Tea State Model

```
type AppState struct {
Messages []Message
ActivePersona string
InputBuffer string
Loading bool
Streaming bool
FocusMode string
ActiveModal string
Theme Theme
}
```

Streaming Pipeline

User Input

↓

Bubble Tea Update()

↓

Async tea.Cmd

↓

HTTP Stream Request

↓

Chunked Response

↓

Token Parser

↓

Viewport Renderer

↓

Lipgloss Styled Output

Event Flow

Prompt Submission

1. User presses A
2. Steam Input sends Enter
3. Bubble Tea catches tea.KeyMsg
4. Input validated
5. Async request launched
6. Streaming begins
7. Viewport updates incrementally

Cancel Generation

1. User presses B
2. ESC event triggered
3. Active request context canceled
4. Stream closed
5. UI state reset

Deployment Strategy

Packaging

- static Go binary
- AppImage support
- tar.gz release
- optional Flatpak

Launch Flow

!/bin/bash

alacritty --config-file ~/.config/alacritty/llm-term.toml -e ./llm-tui

Steam Launch Option

gamescope -W 1280 -H 800 -f -- %command%

Suggested Repository Structure

/llm-term

/cmd

/internal

/ui

/state

/steaminput

/llm

/renderer

/config

/storage

/assets

/themes

/scripts

/docs

Theme System

Theme Config Example

[theme]

background = "#000000"

foreground = "#00FFAA"

accent = "#FF00AA"

border = "#00FFFF"

Accessibility Requirements

- full controller operation
- scalable fonts
- colorblind-safe themes
- configurable bindings
- reduced animation mode

Performance Targets

Metric	Target
Cold Launch	< 2 seconds
Input Latency	< 16ms
RAM Usage	< 350MB
Streaming Delay	< 100ms
FPS	60fps terminal refresh

Security Considerations

Local Security

- encrypted session exports
- no telemetry by default
- sandboxed config loading

Remote Security

- TLS-only API communication
- secure token storage
- provider isolation

QA Strategy

Unit Tests

- reducers
- state transitions
- token parser

Integration Tests

- Ollama connectivity
- Steam Input behavior
- rendering consistency

UX Testing

- handheld ergonomics
- controller-only navigation
- long-session stability

Development Epics

Epic 1 — Core TUI Framework

Goal:

Establish the Bubble Tea foundation and rendering system.

Stories:

- Create root application model
- Implement viewport system
- Build textarea input
- Add async command handling
- Add responsive layout engine

Deliverables:

- functional terminal shell
- keyboard input
- rendering pipeline

Epic 2 — Steam Deck Integration

Goal:

Achieve native-feeling SteamOS interaction.

Stories:

- create Steam Input profile
- build launch wrapper
- test gamescope behavior
- optimize fullscreen rendering
- handle SteamOS keyboard bursts

Deliverables:

- stable Game Mode launch
- full controller support

Epic 3 — LLM Networking Layer

Goal:

Implement streaming inference architecture.

Stories:

- Ollama client
- OpenAI provider adapter
- streaming token parser
- cancellation support
- retry logic

Deliverables:

- live token streaming
- provider abstraction

Epic 4 — Persistence & Export

Goal:

Add long-term workflow features.

Stories:

- save conversations
- export markdown
- load sessions
- create config loader
- theme management

Deliverables:

- persistent workflows

Epic 5 — UX Polish & Effects

Goal:

Create immersive cyberpunk presentation.

Stories:

- animated headers
- ANSI effects
- chroma themes
- transition animations

- radial menus

Deliverables:

- polished immersive interface

Sprint Plan

Sprint 1

- bootstrap repo
- configure Bubble Tea
- create viewport
- build input system

Sprint 2

- implement Steam Input mappings
- create launch wrapper
- validate gamescope support

Sprint 3

- connect Ollama backend
- implement streaming
- add loading indicators

Sprint 4

- persistence layer
- session export
- markdown support

Sprint 5

- theme engine
- animation system
- performance optimization

Sprint 6

- QA hardening
- packaging
- release candidate

MVP Scope

The MVP includes:

- fullscreen TUI
- local Ollama support
- streaming responses
- Steam Deck controls
- session persistence
- configurable themes

Future Roadmap

Phase 2

- plugin system
- Lua automation
- local vector memory

Phase 3

- multi-agent workflows
- voice input
- multiplayer terminal collaboration

Phase 4

- custom scripting runtime
- embedded shell integration
- autonomous coding agents

Example Bubble Tea Update Pattern

```
func (m Model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
switch msg := msg.(type) {
case tea.KeyMsg:
switch msg.String() {
case "enter":
return m, submitPromptCmd(m.Input)
case "esc":
return m, cancelStreamCmd()
case "ctrl+n":
```

m.Messages = nil

return m, nil

}

}

return m, nil

}

Recommended Open Source Dependencies

- bubbletea
- lipgloss
- bubbles
- glamour
- ollama
- chroma
- alacritty

Final Vision

The final application should feel like:

- a cyberdeck operating system
- a handheld AI workstation
- a modern ANSI terminal
- a console-native developer environment

The experience should blur the line between:

terminal,

game UI,

and AI operating system.